Production-Ready Multi-Service Deployment: System Specification

Table of Contents

1. Executive Summary . . . 2
   Technology Stack . . . 3
2. Architecture Overview . . . 3
   Service Summary . . . 5
3. End-to-End Pipeline . . . 6
   Step-by-Step Walkthrough . . . 7
4. The Orchestration Engine — orchestrate.py . . . 8
   4.1 Configuration System (Section 1, Lines 31–141) . . . 8
   4.2 Template Rendering Pipeline (Section 2, Lines 144–238) . . . 9
   4.3 Generator Functions & File Manifest (Sections 3–11) . . . 11
      Complete File Manifest (29 files) . . . 12
   4.4 CLI Interface (Section 12, Lines 1140–1237) . . . 13
5. Template System . . . 14
   5.1 Marker Syntax & Convention . . . 14
   5.2 API Template Anatomy (templates/api/app.py) . . . 15
   5.3 Worker Template (templates/worker/app.py) . . . 17
   5.4 Test Templates . . . 17
   5.5 Migration Template . . . 18
6. Docker Service Architecture . . . 18
   6.1 Service Dependency Graph . . . 18
   6.2 PostgreSQL — Data Layer . . . 19
   6.3 Redis — Cache & Message Broker . . . 19
   6.4 FastAPI API — Application Layer . . . 20
   6.5 Celery Worker — Background Processing . . . 20
   6.6 Nginx — Reverse Proxy . . . 20
   6.7 Prometheus — Metrics Collection . . . 20
   6.8 Grafana — Visualization . . . 21
   6.9 Multi-Stage Dockerfile Pattern . . . 21
   6.10 Network & Volume Topology . . . 22
7. Data Flow Pipelines . . . 22
   7.1 HTTP Request Lifecycle . . . 23
   7.2 Cache-Aside Pattern . . . 24
   7.3 Background Task Flow . . . 25
   7.4 Monitoring Pipeline . . . 26

8. Environment & Configuration Reference . . . 27
9. Development Workflow . . . 28
Typical Workflow . . . 28
Development Overrides . . . 29
Makefile Targets . . . 29
10. Security Architecture . . . 30
Non-Root Containers . . . 30
Network Isolation . . . 30
Environment Variable Management . . . 30
Nginx Security Headers . . . 30
CORS Middleware . . . 31
11. Implementation Roadmap . . . 31
Appendix A: Complete Project File Tree . . . 31
Appendix B: Service URLs Quick Reference . . . 33
Appendix C: Troubleshooting Guide . . . 33
Common Issues . . . 33
Useful Debug Commands . . . 34

Before rendering to PDF: Export the DrawIO diagrams to SVG, then generate the PDF.

```
# Step 1: Export all .drawio → .svg (requires draw.io desktop)
for f in docs/diagrams/*.drawio; do
  /Applications/draw.io.app/Contents/MacOS/draw.io --export --format
svg "$f"
done

# Step 2: Render to PDF
gem install asciidoctor-pdf   # one-time install
asciidoctor-pdf -a imagesdir=diagrams docs/spec.adoc

# Or render to HTML
asciidoctor docs/spec.adoc
```

Alternatively, open `.drawio` files in VS Code with the "Draw.io Integration" extension or at draw.io.

1. Executive Summary

This project is a **production-grade, containerized Task Management REST API** built with FastAPI, PostgreSQL, Redis, and Celery — orchestrated via Docker Compose with Nginx reverse proxying and Prometheus/Grafana monitoring.

The entire project is driven by a single source of truth: `orchestrate.py` (~1,250 lines). This one file

holds all configuration, generates 29 project files from templates and inline generators, and provides a CLI to manage the Docker stack.

Key design principles:

- **Single source of truth** — change a config value once, it propagates to every file that uses it
- **Two kinds of files** — application code lives in `templates/` (editable with full IDE support); infrastructure config is generated from inline functions
- **Production patterns** — multi-stage Docker builds, non-root containers, health checks, named volumes, restart policies, environment variable management

Technology Stack

Component	Technology	Purpose
API Framework	FastAPI 0.135.1 + Uvicorn 0.41.0	Async REST API
ORM	SQLAlchemy 2.0.48 (async)	Database operations
Validation	Pydantic v2 + pydantic-settings 2.13.1	Request/response schemas, config
Database	PostgreSQL 16 (asyncpg 0.31.0)	Persistent storage
Cache / Broker	Redis 7 (redis-py 7.2.1)	Caching + Celery message broker
Task Queue	Celery 5.6.2	Background job processing
Reverse Proxy	Nginx 1.27	Traffic routing, security headers
Monitoring	Prometheus + Grafana	Metrics collection + dashboards
Migrations	Alembic 1.18.4	Database schema versioning
Testing	pytest 8.3.5 + httpx 0.28.1	Async API testing
Containerization	Docker + Docker Compose	Multi-service orchestration

2. Architecture Overview

The system comprises 7 **Docker services** running on a shared bridge network, communicating over internal DNS.

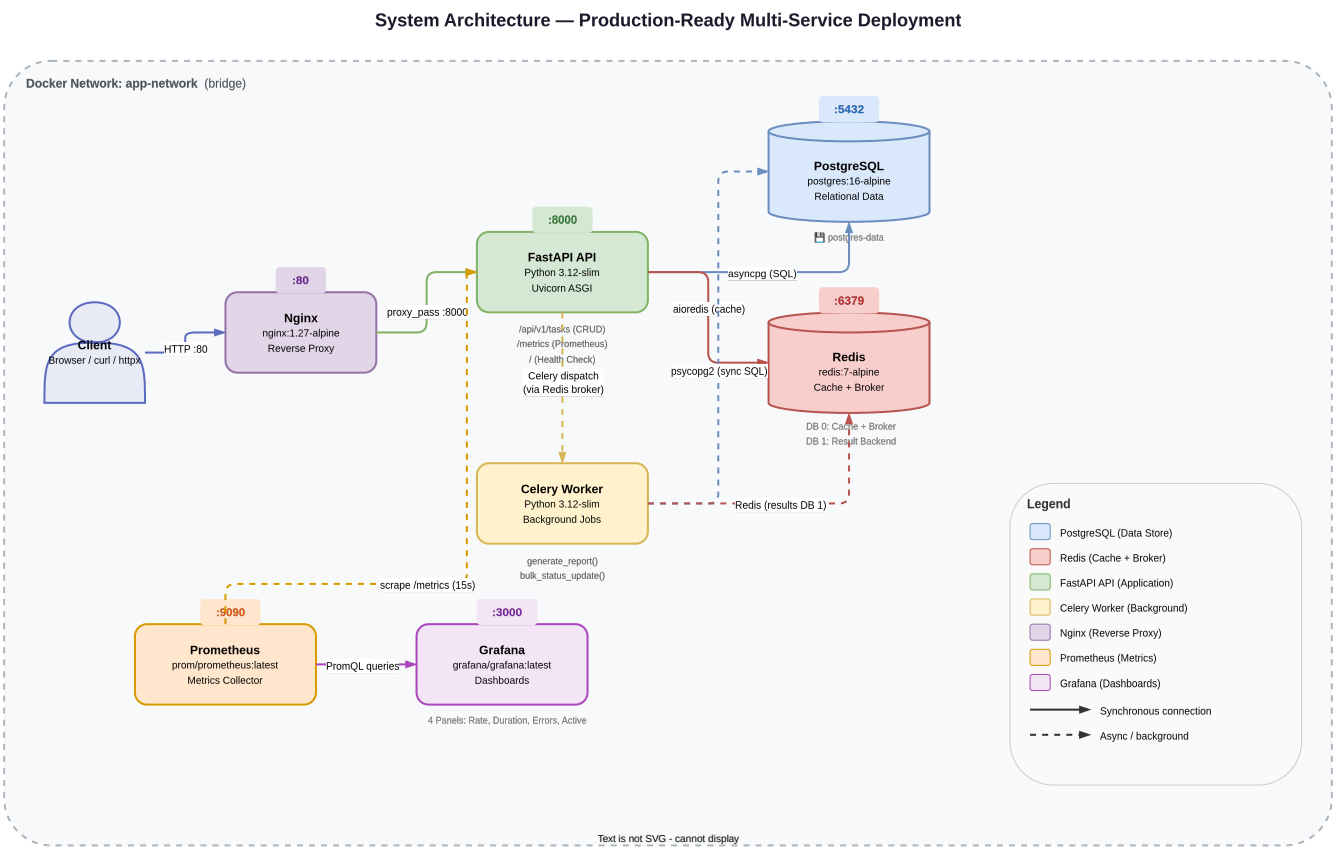

Figure 1. Architecture Overview — All 7 services, ports, and connections

Architecture Overview (ASCII fallback)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                              │
│                                                                                                                              │
│                                                                                                                              │
```

```
 │                └──────────────────────────────┘      │    :6379    │  dispatch
(Celery)      │
 │                        ^             │    redis:7  │──────────────┐                │
 │                        │             └──────────────────────────┘         │
 │
 │                        │   SQL (psycopg2)          ┌──────────────v──────────────┐
 │
 │                        └──────────────────────────────────────────────────────────│ Celery
Worker  │        │
 │                                                    │  (Background)   │        │
 │                                                    └──────────────────────────────┘
 │
 │                                                                                  │
 │    ┌──────────────────────────┐          ┌──────────────────────────┐
 │
 │    │   Prometheus    │────scrape──>  API :8000   │   /metrics endpoint          │
 │    │   :9090         │          └──────────────────────────┘
 │
 │    └──────────────┬──────────────┘
 │
 │           │ PromQL                                                          │
 │    ┌──────────────v──────────────┐
 │
 │    │    Grafana      │                                                       │
 │    │    :3000        │                                                       │
 │    └──────────────────────────┘
 │
 └──────────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────────────────────────────┘
```

Service Summary

Service	Image	Port	Role	Health Check
postgres	postgres:16-alpine	5432	Relational data store	`pg_isready -U taskapp`
redis	redis:7-alpine	6379	Cache + message broker	`redis-cli ping`
api	Custom (Python 3.12-slim)	8000	FastAPI REST API	HTTP GET `localhost:8000/`
worker	Custom (Python 3.12-slim)	—	Celery background worker	`celery -A app inspect ping`
nginx	nginx:1.27-alpine	80	Reverse proxy	—
prometheus	prom/prometheus:latest	9090	Metrics collector	—
grafana	grafana/grafana:latest	3000	Dashboard visualization	—

3. End-to-End Pipeline

This is the master flow — from editing source code to a running multi-service stack.

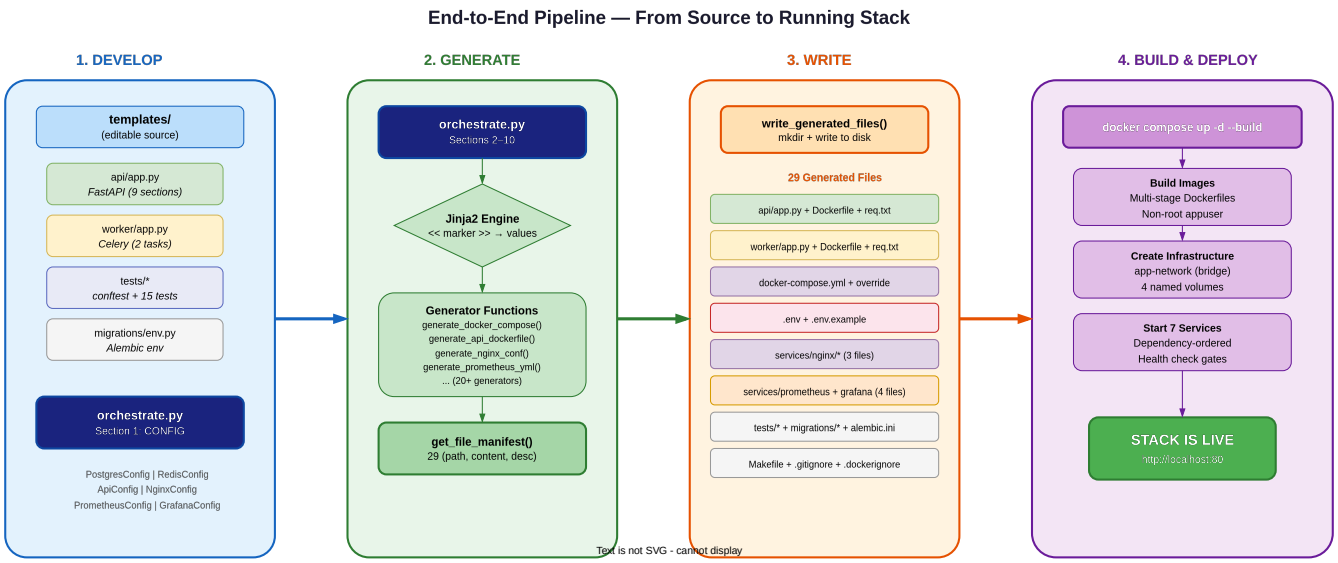

Figure 2. End-to-End Pipeline — From source editing to running stack

End-to-End Pipeline (ASCII fallback)

```
DEVELOP                    GENERATE                      BUILD & DEPLOY
┌──────────────────────────────────────┐
┌──────────────────────────────────────────────────────┐
┌──────────────────────────────────────────────────────────────────┐
│                  │     │                       │     │
│
│   Edit templates/ │     │   orchestrate.py      │     │   Docker Compose
│
│   ──────────────────────────── │     │   ──────────────────────────────────
│     │   ──────────────────────────────────────────     │
│                  │     │                       │     │
│
│   templates/      │     │   1. Load CONFIG      │     │   1. Build images
│
│     api/app.py    │ ───> │   2. Render templates  │ ───> │      (multi-stage)
│
│     worker/app.py │     │      via Jinja2       │     │   2. Create network
│
│     tests/*       │     │   3. Run generators   │     │      (app-network)
│
│                  │     │   4. Write 29 files   │     │   3. Create volumes
│
│   ────── OR ──────      │     │      to disk          │     │      (postgres-data,
etc.)   │
│                  │     │                       │     │   4. Start services
│
│   Edit CONFIG     │     │   CLI: generate / up  │     │      (dependency order)
│
```

```
│   (orchestrate.py  │──────> │                        │──────> │   5. Health checks pass
│
│    Section 1)      │      │                        │      │   6. Stack is live
│
│                    │      │                        │      │
│
└──────────────────────────────────┘
└──────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────┘
                                  │
                                  │  Writes 29 files:
                                  v


┌──────────────────────────────────────────────────────────────────────────────┐
              │   api/app.py          api/Dockerfile   │
              │   api/requirements.txt                 │
              │   worker/app.py       worker/Dockerfile │
              │   worker/requirements.txt              │
              │   docker-compose.yml                   │
              │   docker-compose.override.yml          │
              │   .env  .env.example                   │
              │   services/nginx/*                     │
              │   services/prometheus/*                │
              │   services/grafana/*                   │
              │   migrations/*                         │
              │   tests/*                              │
              │   Makefile  .gitignore  .dockerignore  │
              │   scripts/healthcheck.sh               │
              │   alembic.ini                          │

└──────────────────────────────────────────────────────────────────────────────┘
```

Step-by-Step Walkthrough

1. **Developer edits** `templates/api/app.py` (application code) or `CONFIG` in `orchestrate.py` (configuration)
2. **Developer runs** `python orchestrate.py up` (or `generate` for files only)
3. **orchestrate.py** loads the `CONFIG` object (Section 1)
4. **Template rendering** — Jinja2 processes `templates/*.py` files, replacing `<< marker >>` syntax with CONFIG values
5. **Generators execute** — inline functions produce Dockerfiles, docker-compose.yml, nginx configs, monitoring configs, etc.
6. **File manifest** — `get_file_manifest()` assembles all 29 (path, content, description) tuples
7. **Disk I/O** — `write_generated_files()` creates directories and writes all files
8. **Docker Compose** — `docker compose up -d --build` builds images and starts the 7-service stack
9. **Health checks** — Docker verifies each service is healthy before dependents start

10. **Stack is live** — API accessible at `http://localhost:80` via Nginx

4. The Orchestration Engine — orchestrate.py

`orchestrate.py` is a ~1,250-line Python script organized into 11 numbered sections (1–8, 10–12; section 9 is intentionally skipped). It is the single source of truth for the entire project.

4.1 Configuration System (Section 1, Lines 31–141)

All configuration is defined as Python `@dataclass` objects composed into a single `ProjectConfig`.

```
ProjectConfig (THE SINGLE SOURCE OF TRUTH)
 ├─── project_name: "taskapp"
 ├─── network_name: "app-network"
 ├─── restart_policy: "unless-stopped"
 │
 ├─── PostgresConfig
 │    ├─── user: "taskapp"
 │    ├─── password: "changeme_use_a_strong_password"
 │    ├─── db: "taskdb"
 │    ├─── host: "postgres"         (must match docker-compose service name)
 │    ├─── port: 5432
 │    ├─── image: "postgres:16-alpine"
 │    ├─── volume_name: "postgres-data"
 │    ├─── @property async_url  →
postgresql+asyncpg://taskapp:...@postgres:5432/taskdb
 │    └─── @property sync_url   →  postgresql://taskapp:...@postgres:5432/taskdb
 │
 ├─── RedisConfig
 │    ├─── host: "redis"
 │    ├─── port: 6379
 │    ├─── db_main: 0               (app cache + Celery broker)
 │    ├─── db_results: 1            (Celery result backend)
 │    ├─── image: "redis:7-alpine"
 │    ├─── volume_name: "redis-data"
 │    ├─── @property url               →  redis://redis:6379/0
 │    ├─── @property celery_broker_url  →  redis://redis:6379/0
 │    └─── @property celery_result_backend → redis://redis:6379/1
 │
 ├─── ApiConfig
 │    ├─── host: "0.0.0.0"
 │    ├─── port: 8000
 │    ├─── debug: false
 │    └─── python_version: "3.12"
 │
 ├─── NginxConfig
 │    ├─── external_port: 80
```

```
│     ├──── image_version: "1.27-alpine"
│     ├──── worker_connections: 1024
│     └──── keepalive_timeout: 65
│
├──── PrometheusConfig
│     ├──── port: 9090
│     ├──── image: "prom/prometheus:latest"
│     ├──── scrape_interval: "15s"
│     ├──── evaluation_interval: "15s"
│     └──── volume_name: "prometheus-data"
│
└──── GrafanaConfig
      ├──── port: 3000
      ├──── image: "grafana/grafana:latest"
      ├──── admin_user: "admin"
      ├──── admin_password: "changeme"
      └──── volume_name: "grafana-data"
```

The `CONFIG` object is instantiated at line 140:

```
CONFIG = ProjectConfig()
```

Change any value here, run `python orchestrate.py generate`, and the new value propagates to every file that references it.

4.2 Template Rendering Pipeline (Section 2, Lines 144–238)

Templates live in `templates/` as real Python files with full IDE support. Config values use custom Jinja2 delimiters.

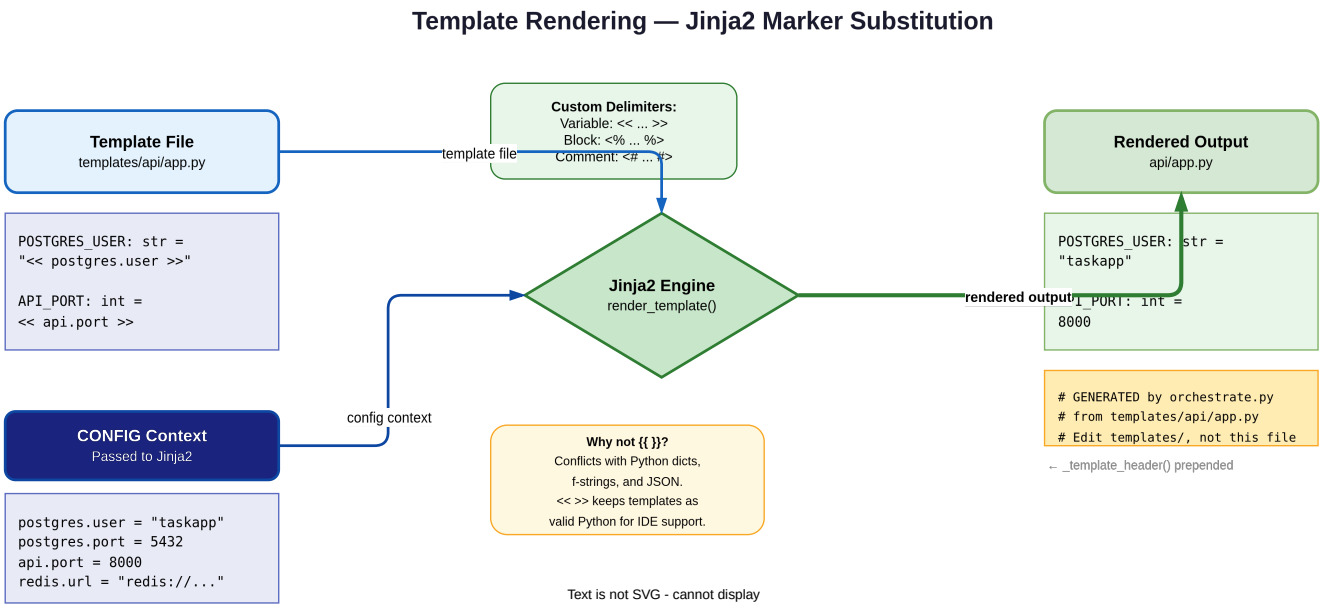

Figure 3. Template Rendering — Jinja2 marker substitution flow

Template Rendering (ASCII fallback)

```
┌──────────────────────────────────────────────────┐
┌──────────────────────────────────────┐
┌──────────────────────────────────────────────┐
│   templates/api/app.py    │      │   Jinja2 Engine    │      │    api/app.py
│
│   ────────────────────────────────────────   │      │
──────────────────────────   │      │   (rendered output)   │
│                           │      │                    │      │
│
│   POSTGRES_USER: str =    │──────>│   Variable:        │──────>│
POSTGRES_USER: str = │
│     "<< postgres.user >>" │      │     << ... >>      │      │     "taskapp"
│
│                           │      │   Block:           │      │
│
│   API_PORT: int =         │      │     <% ... %>      │      │   API_PORT: int =
│
│     << api.port >>        │      │   Comment:         │      │     8000
│
│                           │      │     <# ... #>      │      │
│
└──────────────────────────────────────────────────┘
└──────────────────────────────────────┘
└──────────────────────────────────────────────┘
                                        ^
                                         │
                                  ┌──────────────────┴──────────────────────┐
                                  │   CONFIG context:   │
                                  │   postgres: ...     │
                                  │   redis: ...        │
                                  │   api: ...          │
                                  │   nginx: ...        │
                                  │   prometheus: ...   │
                                  │   grafana: ...      │
                                  │   project: ...      │
                                  └──────────────────────────────────────┘
```

Why custom delimiters? Standard Jinja2 uses `{{ }}` which conflicts with Python dicts, f-strings, and other syntax. The `<< >>` delimiters allow template files to be valid Python that IDEs can parse, lint, and autocomplete.

Key functions:

- `render_template(template_path, config, header_lines)` — core rendering function
- `_template_header(template_path)` — generates the "GENERATED by orchestrate.py" header
- Convenience wrappers: `generate_api_app()`, `generate_worker_app()`, `generate_conftest()`, `generate_test_tasks()`, `generate_tests_init()`, `generate_alembic_env()`

4.3 Generator Functions & File Manifest (Sections 3–11)

Beyond template rendering, `orchestrate.py` has inline generator functions that produce infrastructure files directly from CONFIG values (no Jinja2 templates needed).

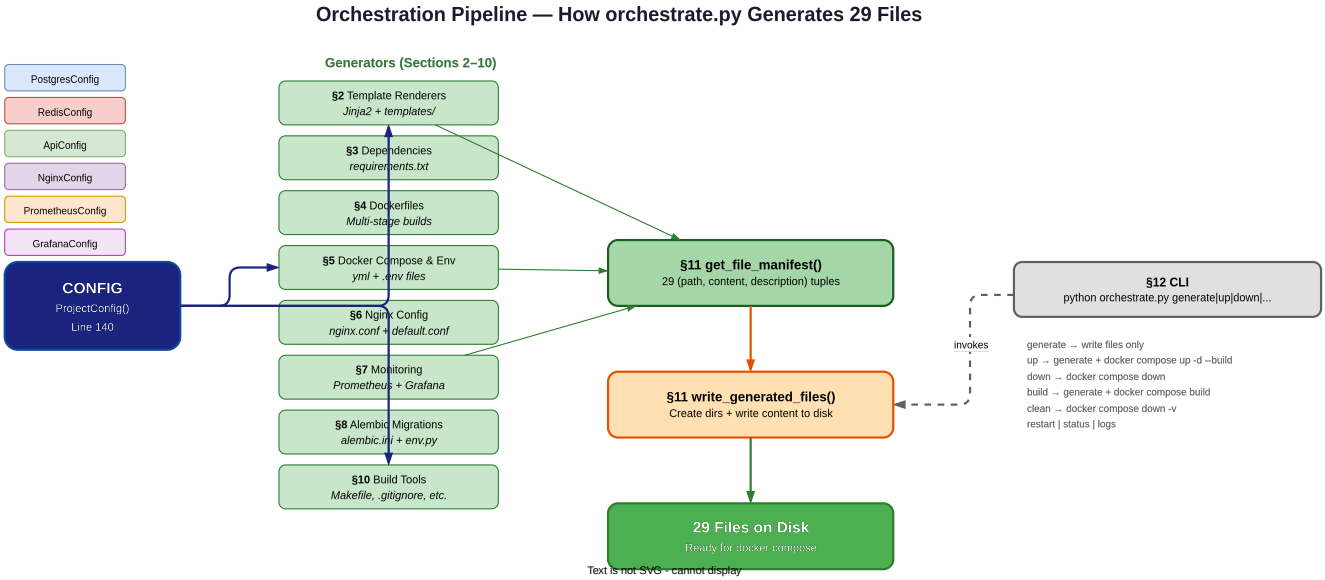

Figure 4. Orchestration Pipeline — CONFIG through generators to 29 files on disk

Orchestration Pipeline (ASCII fallback)

```
orchestrate.py Generator Pipeline
═══════════════════════════════════════════════════════════════

CONFIG ──────> Section 2: Template Renderers ──────> api/app.py, worker/app.py,
          (Jinja2 + templates/)              tests/*, migrations/env.py

CONFIG ──────> Section 3: Dependency Generators ──────> api/requirements.txt,
                                                  worker/requirements.txt

CONFIG ──────> Section 4: Dockerfile Generators ──────> api/Dockerfile,
                                                  worker/Dockerfile,
                                                  services/nginx/Dockerfile

CONFIG ──────> Section 5: Compose Generators ──────> docker-compose.yml,
                                               docker-compose.override.yml,
                                               .env, .env.example

CONFIG ──────> Section 6: Nginx Generators ──────> services/nginx/nginx.conf,
                                              services/nginx/conf.d/default.conf

CONFIG ──────> Section 7: Monitoring Generators ──────>
services/prometheus/prometheus.yml,
                                                  services/grafana/provisioning/*,
                                                  services/grafana/dashboards/*
```

```
CONFIG ──> Section 8: Alembic Generators ──> alembic.ini, migrations/*

CONFIG ──> Section 10: Build Tool Generators ──> Makefile, .gitignore,
                                                 .dockerignore,
                                                 scripts/healthcheck.sh

                              |
                              v
                    Section 11: get_file_manifest()
                    Returns 29 (path, content, description) tuples
                              |
                              v
                    write_generated_files()
                    Creates directories, writes all files to disk
```

Complete File Manifest (29 files)

#	Output Path	Generator	Description
1	`api/app.py`	`generate_api_app()`	FastAPI application
2	`worker/app.py`	`generate_worker_app()`	Celery worker
3	`api/requirements.txt`	`generate_api_requirements()`	API dependencies
4	`worker/requirements.txt`	`generate_worker_requirements()`	Worker dependencies
5	`api/Dockerfile`	`generate_api_dockerfile()`	API multi-stage Dockerfile
6	`worker/Dockerfile`	`generate_worker_dockerfile()`	Worker multi-stage Dockerfile
7	`services/nginx/Dockerfile`	`generate_nginx_dockerfile()`	Nginx Dockerfile
8	`.env`	`generate_env_file()`	Environment variables (secret)
9	`.env.example`	`generate_env_example()`	Env template (safe to commit)
10	`docker-compose.yml`	`generate_docker_compose()`	7-service orchestration
11	`docker-compose.override.yml`	`generate_docker_compose_override()`	Dev overrides (hot reload)
12	`services/nginx/nginx.conf`	`generate_nginx_conf()`	Nginx main config
13	`services/nginx/conf.d/default.conf`	`generate_nginx_default_conf()`	Reverse proxy config
14	`services/prometheus/prometheus.yml`	`generate_prometheus_yml()`	Prometheus scrape config

#	Output Path	Generator	Description
15	`services/grafana/…/datasource.yml`	`generate_grafana_datasource()`	Grafana Prometheus datasource
16	`services/grafana/…/dashboard.yml`	`generate_grafana_dashboard_provider()`	Dashboard provisioning
17	`services/grafana/…/api_dashboard.json`	`generate_grafana_dashboard_json()`	Starter 4-panel dashboard
18	`alembic.ini`	`generate_alembic_ini()`	Alembic migration config
19	`migrations/env.py`	`generate_alembic_env()`	Alembic runtime environment
20	`migrations/script.py.mako`	`generate_alembic_script_mako()`	Migration file template
21	`migrations/README`	`generate_migrations_readme()`	Migrations directory docs
22	`migrations/versions/.gitkeep`	`generate_migrations_versions_gitkeep()`	Directory placeholder
23	`tests/conftest.py`	`generate_conftest()`	Test fixtures
24	`tests/test_tasks.py`	`generate_test_tasks()`	CRUD endpoint tests
25	`tests/init.py`	`generate_tests_init()`	Tests package init
26	`Makefile`	`generate_makefile()`	Development commands
27	`scripts/healthcheck.sh`	`generate_healthcheck_sh()`	Container health check
28	`.gitignore`	`generate_gitignore()`	Git ignore rules
29	`.dockerignore`	`generate_dockerignore()`	Docker ignore rules

4.4 CLI Interface (Section 12, Lines 1140–1237)

`orchestrate.py` exposes 9 subcommands via `argparse`:

Command	Action	Docker Compose Equivalent
`generate`	Render templates + run generators, write 29 files	—
`up`	Generate + `docker compose up -d --build`	`docker compose up -d --build`
`down`	Stop and remove containers	`docker compose down`
`restart`	Restart all services	`docker compose restart`
`clean`	Stop services + delete data volumes	`docker compose down -v`
`build`	Generate + build images (no start)	`docker compose build`

Command	Action	Docker Compose Equivalent
`status`	Show container status	`docker compose ps`
`logs [service]`	Follow logs (all or one service)	`docker compose logs -f [service]`
`test`	Generate + rebuild + run pytest inside the API container	`docker compose exec api pytest`

The `run_docker_compose()` helper auto-detects whether `docker compose` (v2) or `docker-compose` (v1) is available.

5. Template System

5.1 Marker Syntax & Convention

Templates use `<< variable >>` syntax — custom Jinja2 delimiters configured in `orchestrate.py`:

```
template_env = Environment(
    variable_start_string="<<",   # << postgres.user >>
    variable_end_string=">>",
    block_start_string="<%",      # <% if debug %> ... <% endif %>
    block_end_string="%>",
    comment_start_string="<#",    # <# This is ignored #>
    comment_end_string="#>",
)
```

Context passed to templates:

```
context = {
    "postgres": config.postgres,   # PostgresConfig instance
    "redis": config.redis,         # RedisConfig instance
    "api": config.api,             # ApiConfig instance
    "nginx": config.nginx,         # NginxConfig instance
    "prometheus": config.prometheus,
    "grafana": config.grafana,
    "project": config,             # Full ProjectConfig
}
```

Example transformation:

Template (before)	Rendered (after)
`POSTGRES_USER: str = "<< postgres.user >>"`	`POSTGRES_USER: str = "taskapp"`
`API_PORT: int = << api.port >>`	`API_PORT: int = 8000`
`"<< postgres.async_url >>"`	`"postgresql+asyncpg://taskapp:··· @postgres:5432/taskdb"`

5.2 API Template Anatomy (templates/api/app.py)

The API template is a ~660-line single-file FastAPI application organized into 9 sections. Each section only depends on sections above it.

```
templates/api/app.py — 9 Sections
═══════════════════════════════════════════════════════════════════════════

 ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│   Section 1: CONFIGURATION                                                                                    │
│   Settings class, env loading, DATABASE_URL                                                                   │
│   Markers: postgres.*, redis.*, api.*                                                                         │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 2: DATABASE MODELS                                                                                  │
│   Base, TaskStatus enum, TaskPriority enum,                                                                   │
│   Task ORM model (7 columns)                                                                                  │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 3: DATABASE SESSION                                                                                 │
│   AsyncEngine, async_sessionmaker, get_session()                                                              │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 4: SCHEMAS                                                                                          │
│   TaskBase, TaskCreate, TaskUpdate,                                                                           │
│   TaskResponse, TaskListResponse                                                                              │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 5: SERVICES (CRUD)                                                                                  │
│   create_task, get_task, list_tasks,                                                                          │
│   count_tasks, update_task, delete_task                                                                       │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 6: CACHE (Redis)                                                                                    │
│   CacheService class: connect, close,                                                                         │
│   get_cached_task, set_cached_task,                                                                           │
│   invalidate_task_cache                                                                                       │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 7: METRICS (Prometheus)                                                                             │
│   REQUEST_COUNT Counter,                                                                                      │
│   REQUEST_DURATION Histogram,                                                                                 │
│   setup_metrics() + middleware                                                                                │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   Section 8: ROUTES                                                                                           │
│   POST /  GET /  GET /{id}  PATCH /{id}                                                                       │
│   DELETE /{id}  POST /report                                                                                  │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
```

```
────────────────┤
│   Section 9: APP                                  │
│   lifespan, FastAPI instance, CORS,               │
│   router inclusion, health check                  │
└──────────────────────────────────────────────────────────────────────────────────────────────
────────────────┘
```

Key architectural decisions in the API:

- **Async everywhere** — `asyncpg` driver, `AsyncSession`, `async def` routes
- **Dependency injection** — `get_session()` as FastAPI `Depends()` parameter
- **Cache-aside pattern** — Check Redis first on GET, invalidate on UPDATE/DELETE
- **Pydantic v2** — `from_attributes=True` for ORM model serialization
- **Single file** — All code in one `app.py` for simplicity in a containerized context

Task ORM Model:

Column	Type	Constraints	Default
`id`	Integer	Primary key, autoincrement	—
`title`	String(255)	NOT NULL	—
`description`	Text	nullable	NULL
`status`	Enum(TaskStatus)	—	`todos`
`priority`	Enum(TaskPriority)	—	`MEDIUM`
`created_at`	DateTime	—	`func.now()`
`updated_at`	DateTime	nullable	`onupdate=func.now()`

API Endpoints (mounted at `/api/v1/tasks`):

Method	Path	Description	Status
POST	`/`	Create a new task	201
GET	`/`	List tasks with filters + pagination	200
GET	`/{task_id}`	Get single task (cache-aside)	200/404
PATCH	`/{task_id}`	Partial update	200/404
DELETE	`/{task_id}`	Delete task	204/404
POST	`/report`	Trigger async Celery report	202

Additionally, `GET /` at the root serves as a health check returning `{"status": "ok"}`, and `GET /metrics` exposes Prometheus metrics.

5.3 Worker Template (templates/worker/app.py)

A ~120-line single-file Celery application with 2 sections.

Section 1: Celery Configuration

```
celery_app = Celery("worker")
celery_app.conf.broker_url = os.getenv("CELERY_BROKER_URL", "<<
redis.celery_broker_url >>")
celery_app.conf.result_backend = os.getenv("CELERY_RESULT_BACKEND", "<<
redis.celery_result_backend >>")
celery_app.conf.task_serializer = "json"
celery_app.conf.result_serializer = "json"
celery_app.conf.accept_content = ["json"]
celery_app.conf.timezone = "UTC"
```

Section 2: Task Definitions

Task	Purpose	Binding
`generate_report(filters)`	Long-running report of tasks matching filters	`bind=True` (for retries)
`bulk_status_update(task_ids, new_status)`	Batch update multiple task statuses	`bind=True`

> The worker uses **synchronous** SQLAlchemy with `psycopg2-binary` — not `asyncpg`. Celery is process-based and does not run an async event loop. This is why `worker/requirements.txt` differs from `api/requirements.txt`.

5.4 Test Templates

`templates/tests/conftest.py` — Shared pytest fixtures:

Fixture	Purpose
`setup_database`	Create/drop tables before and after each test
`db_session`	Provide a test database session
`client`	Async HTTP client with overridden `get_session` dependency
`sample_task`	Create and return a sample task for reuse

`templates/tests/test_tasks.py` — 15 test cases across 5 groups:

- **CREATE** (3 tests): success, missing title (422), default values
- **LIST** (4 tests): empty, returns all, filter by status, filter by priority
- **GET** (2 tests): success, not found (404)

- **UPDATE** (3 tests): success, partial update, not found (404)
- **DELETE** (3 tests): success, not found (404), idempotent (204 → 404)

5.5 Migration Template

`templates/migrations/env.py` — Alembic runtime environment that imports `Base` from `api/app.py` and uses `<< postgres.async_url >>` as the connection string. Supports both offline and online migration modes.

6. Docker Service Architecture

6.1 Service Dependency Graph

Docker Compose ensures services start in the correct order using `depends_on` with `condition: service_healthy`.

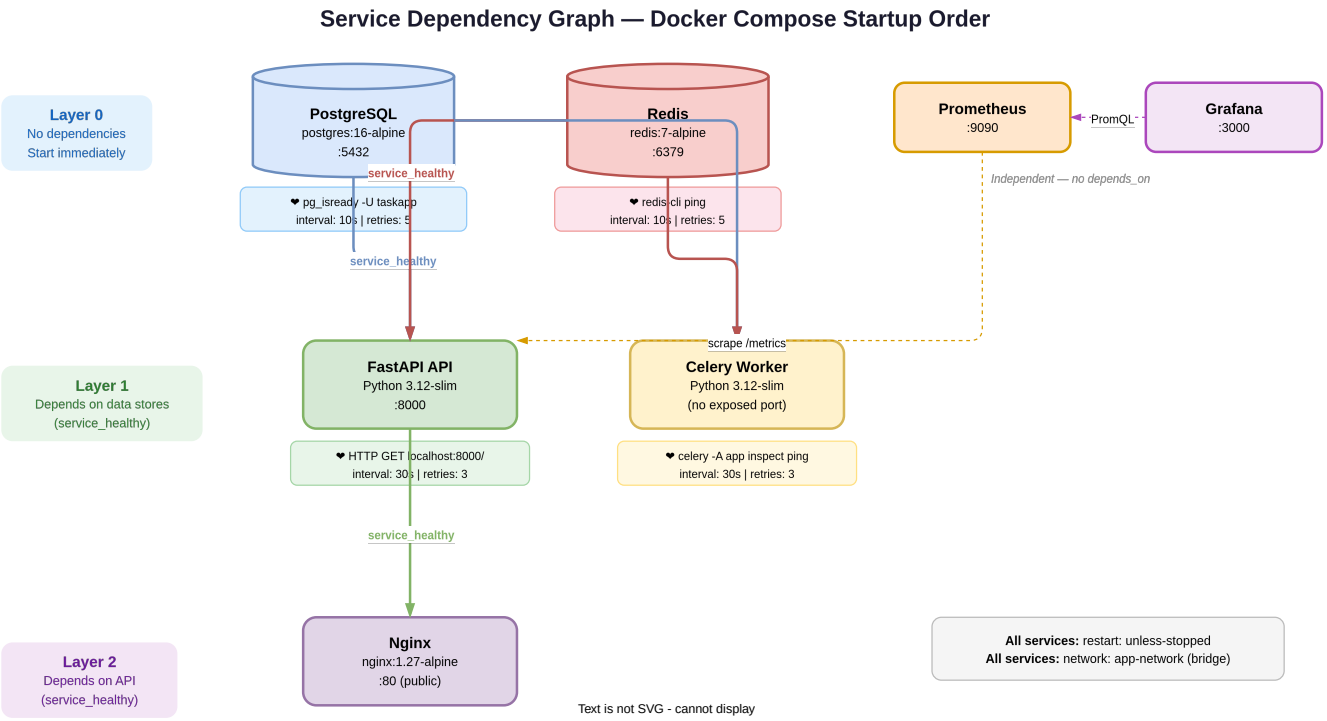

Figure 5. Service Dependency Graph — Startup order with health check gates

Service Dependency Graph (ASCII fallback)

```
                    STARTUP ORDER
                    ═══════════════════════════


  Layer 0 (no dependencies — start immediately)
   ┌──────────────────────────────┐   ┌──────────────────────────────┐
┌──────────────────────────────┐   ┌──────────────────────────────┐
   │  PostgreSQL    │   │     Redis      │   │  Prometheus    │   │    Grafana     │
   │  :5432         │   │     :6379      │   │  :9090         │   │    :3000       │
   └──────────────┬───────────────┘   └──────────────┬───────────────┘
└──────────────────────────────┘   └──────────────────────────────┘
```

```
          |                    |
          | service_healthy    | service_healthy
          |  (pg_isready)      |  (redis-cli ping)
         v                    v
Layer 1 (depends on postgres + redis healthy)
 ┌──────────────────────────┐   ┌──────────────────────────┐
 |   API        |   |   Worker     |
 |   :8000      |   |   (no port)  |
 └──────────────┴──────────────┘   └──────────────────────────┘
          |
          | service_healthy
          |  (HTTP GET :8000/)
         v
Layer 2 (depends on API healthy)
 ┌──────────────────────────┐
 |    Nginx     |
 |    :80       |
 └──────────────────────────┘
```

Startup timeline:

1. PostgreSQL and Redis start first (no dependencies)
2. Health checks run: `pg_isready -U taskapp` every 10s (5 retries) and `redis-cli ping` every 10s (5 retries)
3. Once both are healthy → API and Worker containers start
4. API health check: HTTP GET to `localhost:8000/` every 30s (3 retries)
5. Once API is healthy → Nginx starts and begins proxying traffic
6. Prometheus and Grafana start independently in parallel with everything else

6.2 PostgreSQL — Data Layer

- **Image:** `postgres:16-alpine`
- **Volume:** `postgres-data:/var/lib/postgresql/data` (named, persistent)
- **Env vars from `.env`:** `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`
- **Health check:** `pg_isready -U taskapp` (interval: 10s, timeout: 5s, 5 retries)
- **Used by:** API (asyncpg async), Worker (psycopg2 sync), Alembic (migrations)

6.3 Redis — Cache & Message Broker

- **Image:** `redis:7-alpine`
- **Volume:** `redis-data:/data` (named, persistent)
- **DB 0:** Application cache + Celery broker
- **DB 1:** Celery result backend

- **Health check:** `redis-cli ping` (interval: 10s, timeout: 5s, 5 retries)
- **Used by:** API (cache-aside reads/writes), Celery (task dispatch and results)

6.4 FastAPI API — Application Layer

- **Build context:** `./api` (Dockerfile + app.py + requirements.txt)
- **Env file:** `.env` (all POSTGRES_*, REDIS_URL, CELERY_*, API_*, DEBUG)
- **Internal port:** 8000 (not exposed to host in production — behind Nginx)
- **Dev override:** port 8000 exposed, `--reload` flag, volume-mounted `app.py`
- **Health check:** `python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/')"` (interval: 30s)
- **Key endpoints:** `GET /` (health), `/api/v1/tasks/*` (CRUD), `/metrics` (Prometheus)

6.5 Celery Worker — Background Processing

- **Build context:** `./worker` (Dockerfile + app.py + requirements.txt)
- **Env file:** `.env` (CELERY_*, POSTGRES_*)
- **No exposed ports** — communicates only via Redis broker
- **Health check:** `celery -A app inspect ping` (interval: 30s, timeout: 10s)
- **CMD:** `celery -A app worker --loglevel=info`
- **Tasks:** `generate_report`, `bulk_status_update`

6.6 Nginx — Reverse Proxy

- **Build context:** `./services/nginx`
- **External port:** `80:80` (the only public-facing port)
- **Upstream:** `api:8000` (Docker internal DNS)
- **Configuration:**
  - `nginx.conf` — main config (worker_processes auto, keepalive 65s)
  - `conf.d/default.conf` — reverse proxy rules, security headers (TODO)

6.7 Prometheus — Metrics Collection

- **Image:** `prom/prometheus:latest`
- **Port:** `9090:9090`
- **Volume:** `prometheus-data:/prometheus` + config mounted read-only
- **Scrape config:**
  - Job `fastapi` → targets `api:8000`, path `/metrics`, interval `15s`

- Job `prometheus` → self-scrape at `localhost:9090`

6.8 Grafana — Visualization

- **Image:** `grafana/grafana:latest`
- **Port:** `3000:3000`
- **Volume:** `grafana-data:/var/lib/grafana` + provisioning mounted read-only
- **Env file:** `.env` (GF_SECURITY_ADMIN_USER, GF_SECURITY_ADMIN_PASSWORD)
- **Auto-provisioned:** Prometheus datasource (with explicit `uid: prometheus`) + starter 4-panel dashboard

> The datasource provisioning YAML must include `uid: prometheus` to match the dashboard JSON's datasource references. Without this explicit UID, Grafana auto-generates a random UID and the dashboard panels show "No data".

Starter dashboard panels:

Panel	PromQL Expression
Request Rate (req/sec)	`rate(http_requests_total[5m])`
Request Duration (p95)	`histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))`
Error Rate (4xx + 5xx)	`rate(http_requests_total{status_code=~"[45].."}[5m])`
Active Requests (gauge)	`http_requests_total`

6.9 Multi-Stage Dockerfile Pattern

Both API and Worker Dockerfiles follow the same multi-stage pattern:

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│   STAGE 1: Builder                                  │
│   ──────────────────────────────────                                                    │
│   FROM python:3.12-slim AS builder                  │
│   WORKDIR /app                                      │
│   COPY requirements.txt .                           │
│   RUN pip install --prefix=/install -r ...          │
│                                                     │
│   (Dependencies installed to /install)              │
│   (Leverages Docker layer caching)                  │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│   STAGE 2: Production                               │
│   ────────────────────────────────────────                                              │
│   FROM python:3.12-slim                             │
│   RUN addgroup --system appgroup &&                 │
```

```
│      adduser --system --group appuser          │
│   WORKDIR /app                                 │
│   COPY --from=builder /install /usr/local      │
│   COPY ./app.py ./app.py                       │
│   USER appuser            ← non-root!          │
│   EXPOSE 8000                                  │
│   HEALTHCHECK ...         ← container health   │
│   CMD ["uvicorn", ...]    ← or celery worker   │
└──────────────
```

7.1 HTTP Request Lifecycle

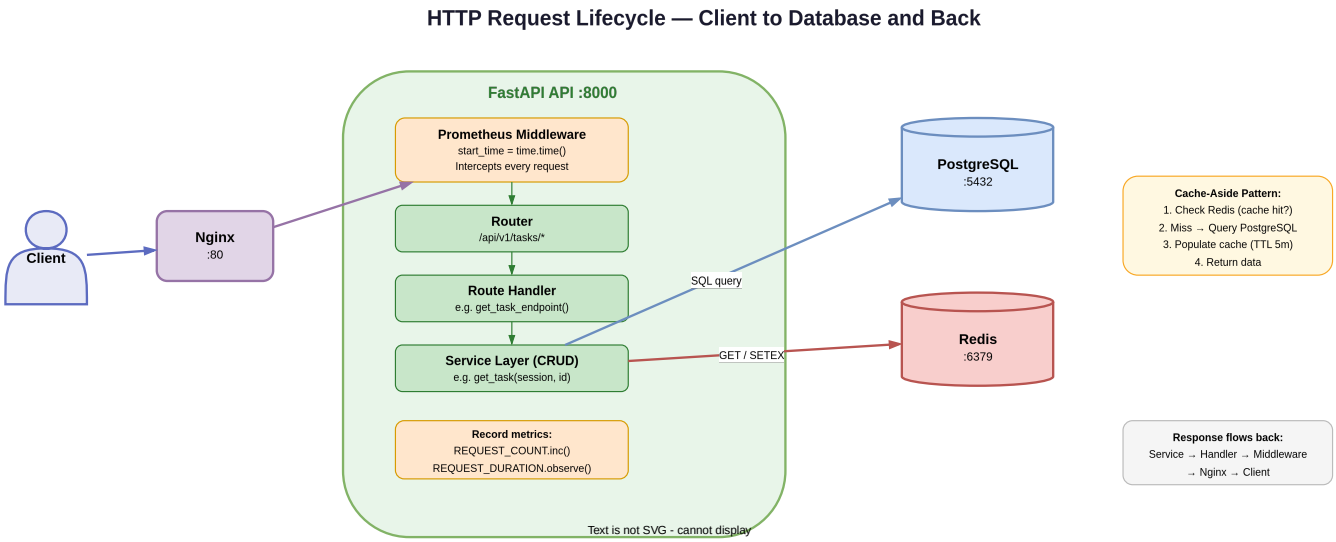

Figure 6. HTTP Request Lifecycle — Client through Nginx and API to data stores

HTTP Request Lifecycle (ASCII fallback)

```
┌──────────────┐       ┌──────────────────────┐
┌──────────────────────────────────────────────────────────────────────────────
┐
│  Client  │──────────>│   Nginx     │──────────>│   FastAPI API
│
│          │       │   :80       │       │   :8000                                  │
│          │       │             │       │                                        │
│          │       │   proxy_    │       │
┌──────────────────────────────────────────────────────────────────────┐      │
│          │       │   pass      │       │   │  Prometheus Middleware          │      │
│          │       │   ──────────────>   │       │   │  - Record start time            │
│
│          │       │             │       │   │  - Call next handler            │      │
│          │       │             │       │   │  - Observe duration (Histogram) │      │
│          │       │             │       │   │  - Increment count (Counter)    │      │
│          │       │             │       │
└──────────────────────┬──────────────────────────────────────────────────────┘      │
│          │       │             │       │              v                          │
│          │       │             │       │
┌──────────────────────────────────────────────────────────────────────┐      │
│          │       │             │       │   │  Route Handler                  │      │
│          │       │             │       │   │  /api/v1/tasks/{id}             │      │
│          │       │             │       │
└──────────────────────┬──────────────────────────────────────────────────────┘      │
│          │       │             │       │              v                          │
│          │       │             │       │
┌──────────────────────────────────────────────────────────────────────┐      │
│          │       │             │       │   │  Service Layer (CRUD)           │      │
│          │       │             │       │
└──────────┬──────────────────────────────────────────┬──────────────────────┘      │
│          │       │             │       │       v                  v              │
```

```
 │         │ <──────── │ <──────────────────── │ <──────── │   PostgreSQL          Redis
Cache          │
 │         │      │            │      │   (read/write)        (cache-aside)      │
 └─────────────────┘       └──────────────────────┘
 ┌──────────────────────────────────────────────────────────────────────────────────────────
┘
```

1. Client sends HTTP request to `localhost:80`
2. Nginx receives the request and `proxy_pass`es to `api:8000`
3. Prometheus middleware intercepts — records start time
4. FastAPI routes to the appropriate handler based on method + path
5. Route handler calls the service layer (CRUD function)
6. Service layer queries PostgreSQL (and optionally Redis cache)
7. Response flows back through middleware (duration recorded) → Nginx → Client

7.2 Cache-Aside Pattern

Used in `GET /api/v1/tasks/{task_id}`:

```
 ┌──────────────────────────────────────────────────────────────────────────────────────────
──────────────────────────┐
 │   GET /api/v1/tasks/{task_id}                          │
 │                                                        │
 │    ┌──────────────────────────────────┐                                    │
 │    │ Check Redis      │──────── HIT ───> Return cached data    │
 │    │ cache_service.   │                                  │
 │    │ get_cached_task  │                                  │
 │    └──────────────────┬───────────────┘                                    │
 │            │                                           │
 │           MISS                                         │
 │            │                                           │
 │            v                                           │
 │    ┌──────────────────────────────────┐                                    │
 │    │ Query PostgreSQL │                                  │
 │    │ get_task()       │                                  │
 │    └──────────────────┬───────────────┘                                    │
 │            │                                           │
 │            ├──────── NOT FOUND ───> Raise 404               │
 │            │                                           │
 │            v                                           │
 │    ┌──────────────────────────────────┐                                    │
 │    │ Populate cache   │    TTL = 300s (5 minutes)          │
 │    │ cache_service.   │    Key = "task:{id}"               │
 │    │ set_cached_task  │                                  │
 │    └──────────────────┬───────────────┘                                    │
 │            │                                           │
```

```
│            v                                                │
│   Return task data                                          │
└─────────────
```

```
│ with    │      │  {task_id,    │      │            │      │              │      │
│
│ task_id │      │   "accepted"} │      │            │      │  Stores      │      │
│
│         │      │               │      │  Result    │<─────────│  result in   │
│             │
│         │      │               │      │  Backend   │      │  Redis DB 1  │      │
│
│         │      │               │      │  (DB 1)    │      │              │      │
│
└─────────────────┘       └──────────────────────────────┘
└────────────────────────┘       └──────────────────────────────┘
└──────────────────────────┘
```

Key points:

- API dispatches asynchronously — returns `202 Accepted` immediately
- Redis DB 0 serves as the Celery message broker (task queue)
- Redis DB 1 serves as the Celery result backend (task results)
- Worker uses **synchronous** SQLAlchemy (`psycopg2`) — no async event loop
- Worker supports retries via `self.retry()` for transient failures

7.4 Monitoring Pipeline

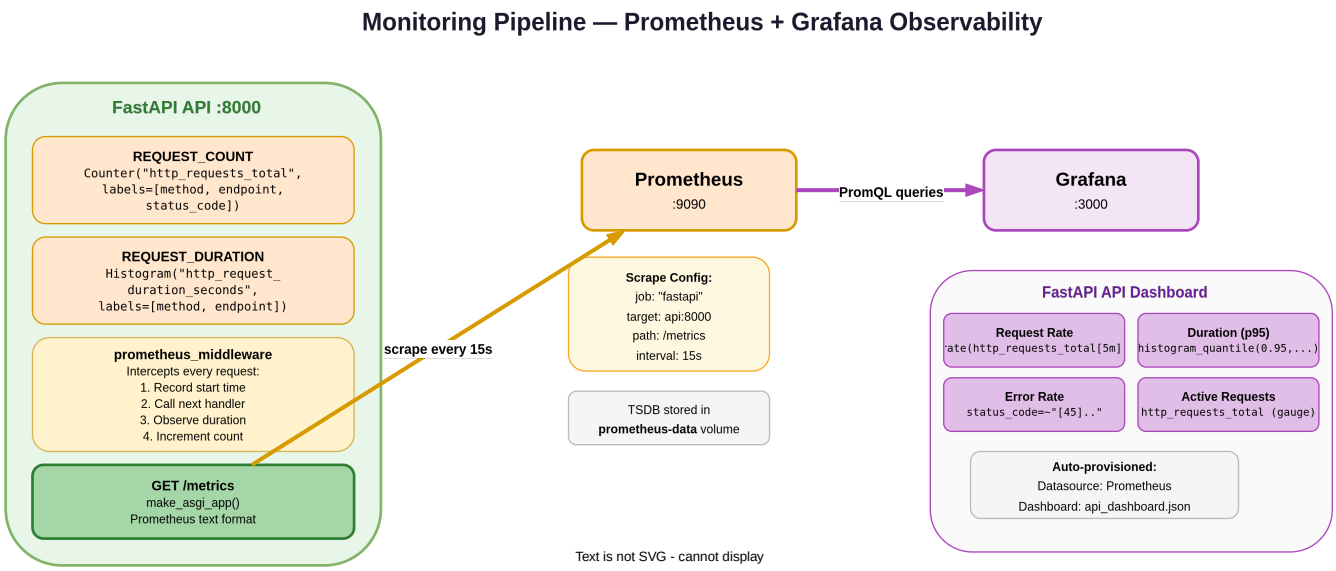

Figure 8. Monitoring Pipeline — Prometheus scraping and Grafana visualization

Monitoring Pipeline (ASCII fallback)

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                              │
│                                                MONITORING PIPELINE                                                           │
│                                                                                                                              │
│    ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
```

```
│
│   │  FastAPI API   │                                                  │
│   │                │                                                  │
│   │  Middleware:   │    Every request:                                 │
│   │   ┌──────────────────────┐ │     - Increment http_requests_total
│
│   │   │ prometheus  │ │       {method, endpoint, status_code}              │
│   │   │ _middleware │ │     - Observe http_request_duration_seconds         │
│   │   └──────────────────────┘ │       {method, endpoint}
│
│   │                │                                                  │
│   │  GET /metrics │──────── Exposes Prometheus text format ──────────┐
│
│   └────────────────────────────┘                                       │
│
│                                                          │          │
│   ┌────────────────────────────┐     scrape every 15s                  │
│
│   │  Prometheus
│<──────────────────────────────────────────────────────────────────────┘
│
│   │  :9090        │                                                  │
│   │               │     Stores time-series data in TSDB               │
│   │  PromQL       │     (prometheus-data volume)                      │
│   └──────────────┬──────────────────┘
│
│          │                                                        │
│          │  PromQL queries                                        │
│         v                                                       |
│   ┌────────────────────────────┐
│
│   │   Grafana     │     Auto-provisioned:                             │
│   │   :3000       │     - Prometheus datasource                       │
│   │               │     - 4-panel dashboard                           │
│   │   ┌──────────────┐   │       1. Request Rate (req/sec)                  |
│   │   │ Panel 1 │   │       2. Request Duration (p95)                  |
│   │   │ Panel 2 │   │       3. Error Rate (4xx+5xx)                    |
│   │   │ Panel 3 │   │       4. Active Requests (gauge)                 |
│   │   │ Panel 4 │   │                                                  |
│   │   └──────────────┘   |                                                  |
│   └────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

8. Environment & Configuration Reference

All environment variables are generated into `.env` from CONFIG and loaded by Docker Compose services via `env_file: .env`.

Variable	Default	Used By	Description
`POSTGRES_USER`	`taskapp`	postgres, api, worker	Database username
`POSTGRES_PASSWORD`	`changeme_use_a_strong_password`	postgres, api, worker	Database password
`POSTGRES_DB`	`taskdb`	postgres, api, worker	Database name
`POSTGRES_HOST`	`postgres`	api, worker	DB hostname (Docker service name)
`POSTGRES_PORT`	`5432`	api, worker	Database port
`REDIS_URL`	`redis://redis:6379/0`	api	Redis connection URL
`CELERY_BROKER_URL`	`redis://redis:6379/0`	api, worker	Celery task broker
`CELERY_RESULT_BACKEND`	`redis://redis:6379/1`	api, worker	Celery result storage
`API_HOST`	`0.0.0.0`	api	API bind address
`API_PORT`	`8000`	api	API listen port
`DEBUG`	`false`	api	Debug mode (true in dev override)
`GF_SECURITY_ADMIN_USER`	`admin`	grafana	Grafana admin username
`GF_SECURITY_ADMIN_PASSWORD`	`changeme`	grafana	Grafana admin password

`.env` is generated and contains secrets. It is listed in `.gitignore` and must never be committed. Use `.env.example` as a reference template.

9. Development Workflow

Typical Workflow

```
# 1. Edit application code in templates
vim templates/api/app.py              # Implement a TODO

# 2. (Optional) Edit configuration
vim orchestrate.py                    # Section 1: CONFIG

# 3. Generate all files and start the stack
python orchestrate.py up              # = generate + docker compose up -d --build

# 4. Test your changes
curl http://localhost/api/v1/tasks    # Test via Nginx
curl http://localhost:8000/api/v1/tasks  # Test API directly (dev override)
```

```
# 5. Run automated tests
make test                           # = docker compose exec api pytest

# 6. Run database migrations
make migrate                        # = docker compose exec api alembic upgrade head

# 7. View logs
python orchestrate.py logs api      # Follow API logs
python orchestrate.py logs worker   # Follow worker logs

# 8. Iterate — go back to step 1
```

Development Overrides

`docker-compose.override.yml` is automatically merged in development:

- **Hot reload** — `api/app.py` is volume-mounted, Uvicorn runs with `--reload`
- **Direct access** — ports 8000 (API), 5432 (Postgres), 6379 (Redis) exposed to host
- **Debug mode** — `DEBUG=true` environment variable

For production-only deployment (no overrides):

```
docker compose -f docker-compose.yml up -d --build
```

Makefile Targets

Target	Command Executed
`make help`	Show all available targets
`make generate`	`python orchestrate.py generate`
`make build`	`python orchestrate.py build`
`make up`	`python orchestrate.py up`
`make down`	`python orchestrate.py down`
`make restart`	`python orchestrate.py restart`
`make logs`	`python orchestrate.py logs`
`make ps`	`python orchestrate.py status`
`make clean`	`python orchestrate.py clean`
`make migrate`	`docker compose exec api alembic upgrade head`
`make test`	`docker compose exec api pytest`
`make shell`	`docker compose exec api /bin/sh`

10. Security Architecture

Non-Root Containers

Both API and Worker Dockerfiles create a dedicated system user:

```
RUN addgroup --system appgroup && adduser --system --group appuser
USER appuser
```

The application process runs as `appuser`, not `root`. This limits the blast radius if the container is compromised.

Network Isolation

- All services communicate on the `app-network` bridge network
- Only Nginx exposes port 80 to the host (production)
- Internal services (API, PostgreSQL, Redis, Worker) are not directly accessible from outside the Docker network in production
- Development overrides expose additional ports for debugging convenience

Environment Variable Management

- Secrets are stored in `.env` (generated, never committed)
- `.env` is listed in `.gitignore`
- `.env.example` provides a safe-to-commit reference template
- Docker Compose loads env vars via `env_file: .env`
- No hardcoded credentials in application code or Dockerfiles

Nginx Security Headers

The following security headers are active on all proxied responses (configured in `generate_nginx_default_conf()`):

- `X-Frame-Options: DENY` — prevents clickjacking
- `X-Content-Type-Options: nosniff` — prevents MIME sniffing
- `X-XSS-Protection: "1; mode=block"` — XSS filter for older browsers
- `Referrer-Policy: strict-origin-when-cross-origin` — limits referrer information

All headers use the `always` directive to ensure they are sent regardless of response status code.

CORS Middleware

FastAPI's `CORSMiddleware` is configured in Section 9 of the API template to control cross-origin request access.

11. Implementation Roadmap

The project is organized into 9 implementation phases, all of which are now **complete**.

Phase	Description	File(s)	Status
1	API Core — Config, Models, Schemas, CRUD, Routes, App	`templates/api/app.py`	▯ Complete
2	Database Session (done as part of Phase 1)	`templates/api/app.py` Sec 3	▯ Complete
3	Redis Caching Layer — cache-aside pattern	`templates/api/app.py` Sec 6	▯ Complete
4	Celery Background Worker — task definitions	`templates/worker/app.py`	▯ Complete
5	Dockerfiles (pre-built by orchestrate.py)	`api/Dockerfile`, `worker/Dockerfile`	▯ Complete
6	Docker Compose (pre-built by orchestrate.py)	`docker-compose.yml`	▯ Complete
7	Prometheus Metrics — Counter, Histogram, middleware	`templates/api/app.py` Sec 7	▯ Complete
8	Nginx Security — headers, proxy config	`orchestrate.py` Sec 6 generators	▯ Complete
9	Testing — fixtures + 15 test cases	`templates/tests/*`	▯ Complete

Implementation order followed: 1 → 2 → 3 → 4 → 7 → 8 → 9 (Phases 5–6 were pre-built)

Appendix A: Complete Project File Tree

```
.
├─── orchestrate.py                          SINGLE SOURCE OF TRUTH (~1,250
lines)
│
├─── templates/                              YOU EDIT THESE (full IDE support)
│    ├─── api/app.py                         FastAPI application (9 sections)
│    ├─── worker/app.py                      Celery worker (2 sections)
│    ├─── tests/
│    │    ├─── conftest.py                    Test fixtures (4 fixtures)
│    │    ├─── test_tasks.py                  Test cases (15 tests)
│    │    └─── __init__.py
```

```
│    └──── migrations/
│        └──── env.py                              Alembic environment
│
├──── api/                                        GENERATED
│    ├──── app.py                                  ← from templates/api/app.py
│    ├──── Dockerfile                              ← from generate_api_dockerfile()
│    └──── requirements.txt                        ← from generate_api_requirements()
│
├──── worker/                                     GENERATED
│    ├──── app.py                                  ← from templates/worker/app.py
│    ├──── Dockerfile                              ← from
generate_worker_dockerfile()
│    └──── requirements.txt                        ← from
generate_worker_requirements()
│
├──── services/
│    ├──── nginx/
│    │    ├──── Dockerfile                          ← from
generate_nginx_dockerfile()
│    │    ├──── nginx.conf                          ← from generate_nginx_conf()
│    │    └──── conf.d/
│    │        └──── default.conf                    ← from
generate_nginx_default_conf()
│    ├──── prometheus/
│    │    └──── prometheus.yml                      ← from generate_prometheus_yml()
│    └──── grafana/
│        ├──── dashboards/
│        │    └──── api_dashboard.json              ← from
generate_grafana_dashboard_json()
│        └──── provisioning/
│            ├──── datasources/
│            │    └──── datasource.yml              ← from
generate_grafana_datasource()
│            └──── dashboards/
│                └──── dashboard.yml               ← from
generate_grafana_dashboard_provider()
│
├──── tests/                                      GENERATED
│    ├──── conftest.py                             ← from templates/tests/conftest.py
│    ├──── test_tasks.py                           ← from
templates/tests/test_tasks.py
│    └──── __init__.py
│
├──── migrations/                                 GENERATED
│    ├──── env.py                                  ← from templates/migrations/env.py
│    ├──── script.py.mako                          ← from
generate_alembic_script_mako()
│    ├──── README                                  ← from
generate_migrations_readme()
│    └──── versions/
│        └──── .gitkeep
```

```
│
├──── scripts/
│     └──── healthcheck.sh                          ← from generate_healthcheck_sh()
│
├──── docker-compose.yml                            ← from generate_docker_compose()
├──── docker-compose.override.yml                   ← from
generate_docker_compose_override()
├──── alembic.ini                                   ← from generate_alembic_ini()
├──── Makefile                                      ← from generate_makefile()
├──── .env                                          ← from generate_env_file() (SECRET)
├──── .env.example                                  ← from generate_env_example()
├──── .gitignore                                    ← from generate_gitignore()
├──── .dockerignore                                 ← from generate_dockerignore()
├──── README.md                                     Quick-start documentation
├──── docs/
│     ├──── spec.adoc                                This specification document
│     ├──── spec.pdf                                 Rendered PDF
│     ├──── BUILD.sh                                 PDF build script
│     └──── diagrams/                                DrawIO architecture diagrams
└──── PROJECT_GUIDE.md                              Step-by-step implementation guide
```

Appendix B: Service URLs Quick Reference

Service	URL	Notes
API (via Nginx)	http://localhost	Production entry point
API (direct)	http://localhost:8000	Dev override only
Health check	http://localhost/	Returns `{"status": "ok"}`
Task CRUD	http://localhost/api/v1/tasks/	REST endpoints (trailing slash required on list endpoint)
Prometheus metrics	http://localhost/metrics	Text format
Prometheus UI	http://localhost:9090	Query interface
Grafana	http://localhost:3000	Login: admin / changeme
PostgreSQL	localhost:5432	Dev override only
Redis	localhost:6379	Dev override only

Appendix C: Troubleshooting Guide

Common Issues

Problem	Solution
`docker compose` command not found	Install Docker Desktop or Docker Engine v20.10+

Problem	Solution
Port 80 already in use	Change `nginx.external_port` in CONFIG, regenerate
PostgreSQL health check failing	Check `docker compose logs postgres` for errors
API can't connect to PostgreSQL	Ensure `POSTGRES_HOST=postgres` (Docker service name, not localhost)
Redis connection refused	Check Redis health: `docker compose exec redis redis-cli ping`
Celery worker not picking up tasks	Verify `CELERY_BROKER_URL` matches between API and Worker
Templates not reflecting changes	Run `python orchestrate.py generate` to re-render
Hot reload not working	Ensure `docker-compose.override.yml` exists (dev mode)
Grafana shows no data	Verify datasource UID matches (`uid: prometheus` in `datasource.yml` must match dashboard JSON). If UID was changed, delete the Grafana volume and recreate. Also check Prometheus is scraping: `curl http://localhost:9090/api/v1/targets`

Useful Debug Commands

```
# Check all service status
python orchestrate.py status

# View logs for a specific service
python orchestrate.py logs api
python orchestrate.py logs worker
python orchestrate.py logs postgres

# Shell into the API container
make shell

# Run a one-off command in the API container
docker compose exec api python -c "from app import settings;
print(settings.DATABASE_URL)"

# Check database connectivity
docker compose exec postgres psql -U taskapp -d taskdb -c "SELECT 1;"

# Check Redis connectivity
docker compose exec redis redis-cli ping

# Inspect Celery worker
docker compose exec worker celery -A app inspect active

# Full reset (destroy all data)
python orchestrate.py clean
```

```
python orchestrate.py up
```